COER UNIVERSITY, ROORKEE

Department of Computer Science & Engineering

Full Stack Web Development

Lab Sheet 6— APIs (Practical)

Full Backend Integration: Node.js + Express.js + REST APIs (No Database)

Objective: Build a complete Node.js + Express REST API from scratch, secure and test it, then consume it from both a plain JavaScript client and a React client — covering the full backend-to-frontend pipeline, without a database.

Q1. Server Setup & Building the REST API

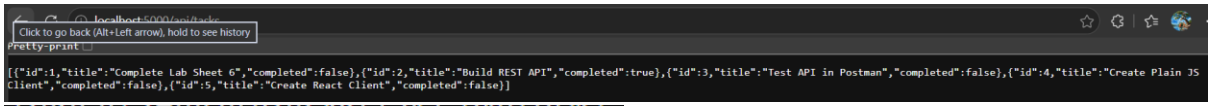
```
const express = require("express");
const cors = require("cors");
require("dotenv").config();
const app = express();
const PORT = process.env.PORT || 5000;
const API_KEY = process.env.API_KEY;
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
// Custom global logger middleware
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`);
  next();
});
// Authentication middleware for write operations only
const authMiddleware = (req, res, next) => {
  const key = req.headers["x-api-key"];
  if (!key || key !== API_KEY) {
    return res.status(401).json({
      error: "Unauthorized",
      message: "Valid x-api-key header is required"
    });  } next(); };
```

```
// In-memory task data
let tasks = [
  { id: 1, title: "Complete Lab Sheet 6", completed: false },
  { id: 2, title: "Build REST API", completed: true },
  { id: 3, title: "Test API in Postman", completed: false },
  { id: 4, title: "Create Plain JS Client", completed: false },
  { id: 5, title: "Create React Client", completed: false }
];
// GET all tasks
app.get("/api/tasks", (req, res) => {
  res.json(tasks);
});
// GET one task
app.get("/api/tasks/:id", (req, res, next) => {
  const id = Number(req.params.id);
  const task = tasks.find((item) => item.id === id);
  if (!task) {
    const error = new Error("Task not found");
    error.status = 404;
    return next(error);
  }
  res.json(task);
});
// POST task
app.post("/api/tasks", authMiddleware, (req, res) => {
  const { title, completed = false } = req.body;
  if (!title) {
    return res.status(400).json({
      error: "Bad Request",
      message: "title is required"
    });
  }
  const newTask = {
    id: tasks.length ? Math.max(...tasks.map((task) => task.id)) + 1 : 1,
    title,
    completed: Boolean(completed)
  };
```

```
  tasks.push(newTask);
  res.status(201).json(newTask);
});
// PUT task
app.put("/api/tasks/:id", authMiddleware, (req, res, next) => {
  const id = Number(req.params.id);
  const index = tasks.findIndex((item) => item.id === id);
  if (index === -1) {
    const error = new Error("Task not found");
    error.status = 404;
    return next(error);
  }
  const { title, completed } = req.body;
  if (title !== undefined) tasks[index].title = title;
  if (completed !== undefined) tasks[index].completed = Boolean(completed);
  res.json(tasks[index]);
});
// DELETE task
app.delete("/api/tasks/:id", authMiddleware, (req, res, next) => {
  const id = Number(req.params.id);
  const index = tasks.findIndex((item) => item.id === id);
  if (index === -1) {
    const error = new Error("Task not found");
    error.status = 404;
    return next(error);
  }
  const deletedTask = tasks.splice(index, 1)[0];
  res.json({
    message: "Task deleted successfully",
    task: deletedTask
  });
});
// 404 handler - keep after all routes
app.use((req, res) => {
  res.status(404).json({
    error: "Route not found",
    message: `No route found for ${req.method} ${req.originalUrl}  });});
```

```
// Centralized error handler - 4 parameters
app.use((err, req, res, next) => {
  console.error(err.stack);
  const status = err.status || 500;
  res.status(status).json({
    error: status === 500 ? "Internal Server Error" : err.message,
    message: err.message
  });
});
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

```
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Server running at http://localhost:5000
```




```
[2026-09-24T07:03:27.205Z] GET /api/tasks
```

Q2. Securing, Testing & Error-Handling the API

Before Enabling CORS

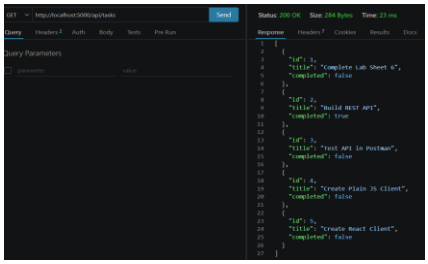
```
Access to fetch at 'http://localhost:5000/api/tasks' from origin 'http://127.0.0.1:5500' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```
index.html:1

After enabling CORS

Task Manager

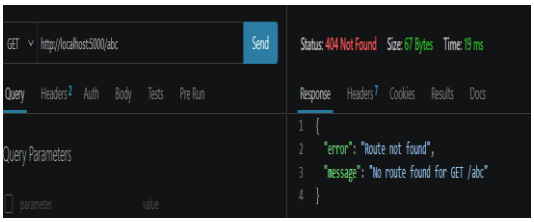
- Complete Lab Sheet 6 - Pending
- Build REST API - Completed
- Test API in Postman - Pending
- Create Plain JS Client - Pending
- Create React Client - Pending

Requests from thunder clients are successful



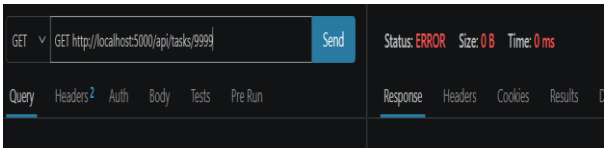
#404 Handler

```
app.use((req, res) => {
    res.status(404).json({
        error: "Route not found"
    }); });
```



#Centralized Error Handler

```
app.use((err, req, res, next) => { console.error(err.stack);
res.status(err.status || 500).json({
error: err.message || "Internal Server Error" }); });
```



Q3. Consuming the API – Plain JavaScript Client

Script.js

```
const API_URL = "http://localhost:5000";
const loading = document.getElementById("loading");
const errorBox = document.getElementById("error");
const taskList = document.getElementById("taskList");
async function loadTasks() {
  loading.textContent = "Loading...";
  errorBox.textContent = "";
  taskList.innerHTML = "";
  try {
    const response = await fetch(`${API_URL}/api/tasks`);
    if (!response.ok) {
      throw new Error(`Request failed with status ${response.status}`);  }
    const tasks = await response.json();
    tasks.forEach((task) => {
      const li = document.createElement("li");
      li.textContent = `${task.title} - ${task.completed ? "Completed" : "Pending"}`;
      taskList.appendChild(li);
    });
  } catch (error) {
    errorBox.textContent =
      "Unable to load tasks. Please make sure the API server is running.";
  } finally {    loading.textContent = "";}
```

```
} loadTasks();
```

index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Task Manager - Plain JavaScript</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      max-width: 700px;

      margin: 40px auto;

      padding: 20px;

    }

    li {

      margin: 10px 0;

      padding: 8px;

      border: 1px solid #ddd;

      border-radius: 5px;

    }

    #error { color: #b00020; }

  </style>

</head>

<body>

  <h1>Task Manager</h1>

  <p id="loading">Loading...</p>

  <p id="error"></p>

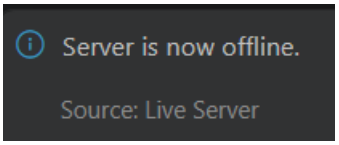
  <ul id="taskList"></ul>



  <script src="script.js"></script>

</body> </html>
```





APP.jx

```
useEffect(() => {

    const loadTasks = async () => {

        setLoading(true);

        setError("");

 try {

            const response = await fetch(`${API_URL}/api/tasks`);

if (!response.ok) {

                const data = await response.json().catch(() => ({}));

                throw new Error(

                    data.message || `Request failed: ${response.status}`

                );}

const data = await response.json();

            setTasks(data);

} catch (err) {

            setError(err.message || "Unable to load tasks.");

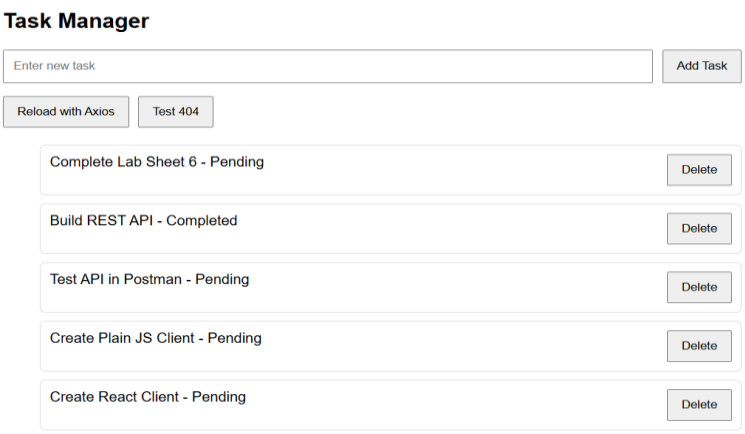
} finally {

            setLoading(false);

        }};

loadTasks();

}, []);
```

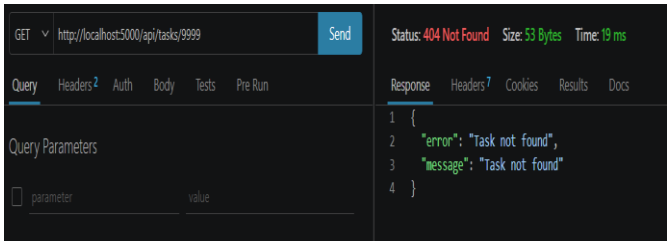


Added Task



Q5. Full-Stack Error Handling, Config & Final Integration

Handle 404 and 401 in React



Task Manager Mini App

